

Medical Image Classification Module

**MedAssistAI
MILESTONE-4**

Kaushika Yadav G

ABSTRACT

MedAssistAI is an AI-powered healthcare assistance platform developed to provide multiple healthcare-related services through a unified web application. The project combines machine learning, artificial intelligence, web technologies, and database services to support symptom-based disease prediction, medical report analysis, AI consultation, personalized recommendations, appointment management, and hospital-related services.

The earlier milestones focused on developing the machine learning components of the system. The symptom-based disease prediction system was developed using structured symptom data, while the medical image classification component was developed using transfer learning with MobileNetV2. The previous milestone established the connection between the trained machine learning model, Flask backend, and React frontend.

Milestone 4 focuses on converting these individual components into an integrated full-stack application. Flask REST APIs were organized into modular routes, MongoDB was integrated for persistent data storage, authentication was implemented, and application modules such as medical reports, appointments, hospitals, AI consultation, symptom analysis, and recommendations were integrated with the frontend.

The milestone also focused on testing the major workflows and preparing the application for deployment.

Keywords: Artificial Intelligence, Machine Learning, Healthcare, React, Flask, MongoDB, Disease Prediction, Medical Report Analysis.

1. INTRODUCTION

Healthcare applications increasingly use artificial intelligence to assist users in accessing health information, analyzing medical data, and obtaining preliminary insights. However, many healthcare applications provide only one isolated functionality.

MedAssistAI was developed as a unified platform combining multiple AI-assisted healthcare functionalities.

The main objective of Milestone 4 was to integrate the components developed during previous milestones into a functional full-stack application.

The system consists of:

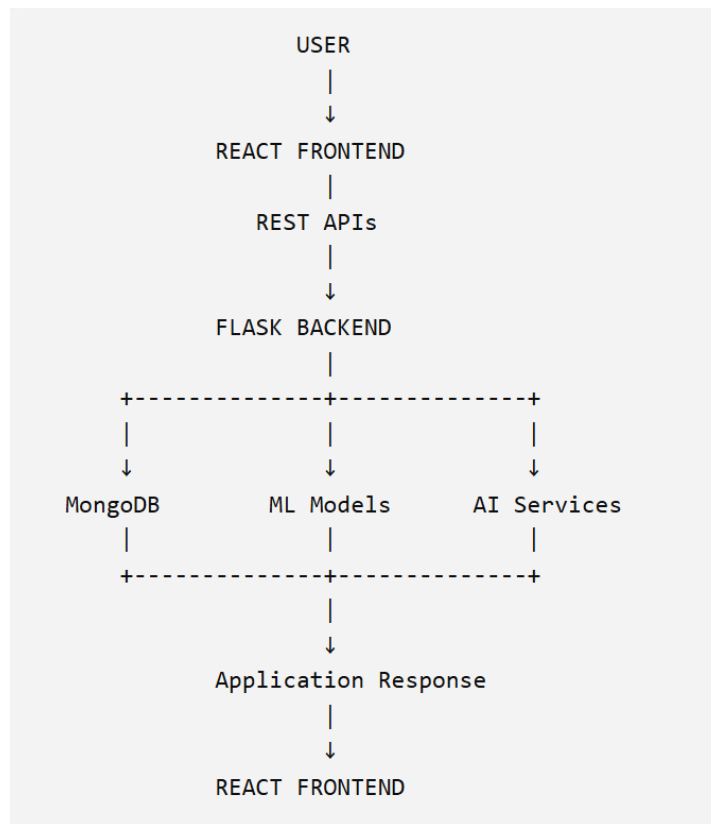
- React frontend
- Flask backend
- MongoDB database
- Machine learning models
- AI services
- Medical report/file management

The symptom prediction component uses a trained disease classification model and supporting preprocessing files.

The project is intended as an **AI-assisted healthcare platform and not a replacement for professional medical diagnosis.**

2. SYSTEM ARCHITECTURE

The MedAssistAI system follows a client-server architecture.



2.1 Frontend

The React frontend provides the user interface for the different healthcare services.

Major sections include:

- AI Medical Consultant
- AI Symptom Analysis
- Medical Report Analysis
- Image Analysis Results
- AI Recommendations
- Hospitals Registered
- Emergency Assistance
- Appointment History
- Total Reports
- User Management
- Settings

2.2 Backend

The Flask backend acts as the central application layer.

The backend was divided into modular route files:

backend/

```
|
|└── routes/
|   ├── auth_routes.py
|   ├── gemini_routes.py
|   ├── user_routes.py
|   ├── report_routes.py
|   ├── appointment_routes.py
|   ├── hospital_routes.py
|   └── prediction_routes.py
|
|└── utils/
|└── uploads/
|└── app.py
|└── db.py
└── requirements.txt
```

This modular architecture makes the system easier to maintain and extend.

2.3 Database

MongoDB was integrated as the persistent database.

It is used for application data such as:

- User information
- Patient records
- Medical reports
- Appointments
- Other application records

The backend initializes the database during application startup.

3. IMPLEMENTATION

3.1 Authentication

User authentication was integrated into the application.

The authentication module provides:

- User registration
- User login
- Authentication tokens
- Protected routes

- User-specific access

Protected APIs use authentication middleware to verify the requesting user before allowing access.

Demo accounts are also seeded for application testing.

3.2 Disease Prediction Integration

The trained disease prediction model developed during earlier milestones was integrated into the Flask backend.

The required model files include:

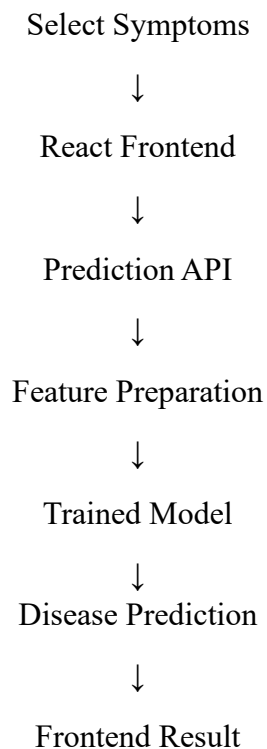
best_decision_tree_model.pkl

disease_label_encoder.pkl

feature_columns.pkl

The model is loaded when the backend starts rather than retrained for every request.

The workflow is:



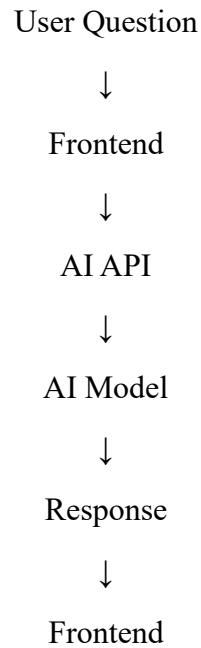
This completes the frontend-to-model prediction pipeline established in Milestone 3.

3.3 AI Medical Consultant

An AI Medical Consultant module was integrated into the frontend.

Users can enter questions or describe symptoms through a conversational interface.

The request is sent to the backend AI API, processed using the configured AI service, and the response is returned to the frontend.

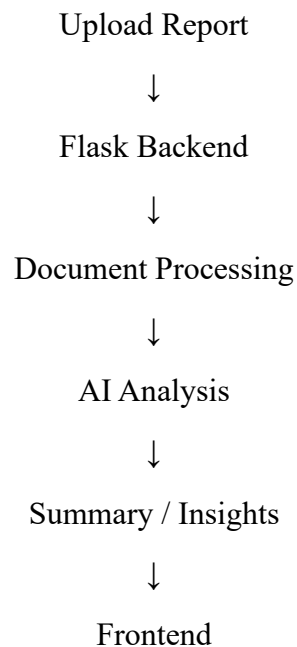


The interface also displays a disclaimer informing users that the AI system is not a doctor.

3.4 Medical Report Analysis

A medical report analysis interface was developed to allow users to upload medical documents.

The workflow is:



A sample medical report was prepared for testing this functionality. The sample is explicitly intended as fictional test data rather than an actual patient medical record.

3.5 Medical Report Management

A separate report management module was implemented using Flask and MongoDB.

The module supports:

- Retrieving reports
- Creating report records
- Uploading files
- Updating report status
- Storing report metadata
- Maintaining report history
- Managing attachments

Uploaded files are stored in an application upload directory with secure filenames.

4. APPLICATION FEATURES

4.1 AI Symptom Analysis

The symptom analysis page provides a user-friendly interface for selecting symptoms.

Additional information can be provided, including:

- Severity
- Duration
- Onset
- Existing diseases
- Allergies
- Current medications

The selected information is sent to the prediction backend and the predicted diseases are displayed on the results panel.

4.2 AI Recommendations

The recommendation module provides personalized general wellness suggestions.

The interface collects information such as:

- Age
- Weight
- Height
- Gender
- Activity level

BMI is calculated from height and weight and displayed along with its category.

The page also provides sections for:

- Diet plan
- Daily calorie information
- Meal recommendations
- Exercise
- Lifestyle recommendations

4.3 Appointment Management

Appointment functionality was integrated into the application.

A dedicated backend route handles appointment-related operations, while the frontend provides an appointment history interface.

This allows appointment information to become part of the user's healthcare record within the application.

4.4 Hospital Module

A hospital module was added to the application.

The backend provides a dedicated hospital API, while the frontend provides a **Hospitals Registered** section.

The module provides a foundation for future functionality such as:

- Hospital search
- Location-based filtering
- Hospital details
- Emergency services
- Doctor availability

4.5 Image Analysis

The project also incorporates the medical image classification work developed during earlier milestones.

The image classification model was developed using MobileNetV2 transfer learning. The implementation includes image preprocessing, model training, validation, testing, and model saving.

The frontend contains an **Image Analysis Results** section for presenting image-analysis-related results.

5. API AND BACKEND VALIDATION

A backend health-check endpoint was implemented:

GET /api/health

It returns:

```
{  
  "status": "healthy",  
  "service": "MedAssistAI Flask Backend"  
}
```

A root endpoint was also implemented:

GET /

which returns:


```
{  
  "success": true,  
  "message": "MedAssistAI Flask API is running"  
}
```

These endpoints were used to verify that the Flask backend was running correctly.

The major API modules include:

Module	API
Authentication	/api/auth
AI Services	/api/gemini
Users	/api/users
Reports	/api/reports
Appointments	/api/appointments
Hospitals	/api/hospital
Prediction	/api/...

6. TESTING AND RESULTS

Testing was performed on the major components of the application.

Backend Testing

The following were tested:

- Flask application startup
- Database connection
- ML model loading
- API availability
- Authentication
- Prediction API
- Report API
- File upload

The disease prediction model and feature files were successfully loaded during backend startup.

Frontend Testing

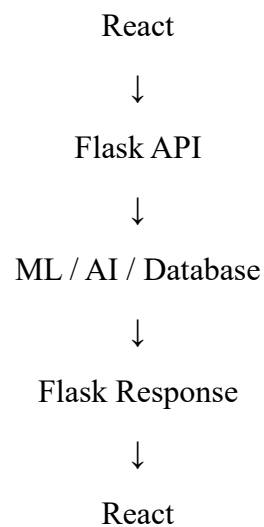
The following interfaces were tested:

- Login
- Dashboard

- AI Consultant
- Symptom Analysis
- Medical Report Analysis
- AI Recommendations
- Reports
- Appointment History
- Hospital section

Integration Testing

The primary integration workflow was verified:



This confirmed that the major application layers could communicate successfully.

7. CHALLENGES AND LIMITATIONS

7.1 Dataset Limitations

The symptom dataset contains a large number of disease classes and significant class imbalance. Earlier analysis identified 189,601 records, 377 symptom features, and 727 disease classes.

This affects prediction reliability and remains an important limitation of the current system.

7.2 AI API Dependency

AI-powered functionality depends on external AI services and therefore requires valid API credentials and network connectivity.

7.3 Demonstration Data

Some application modules use seeded or demonstration data for testing and interface development.

7.4 Medical Safety

The predictions and AI-generated information are intended only as assistance. They should not be considered professional medical diagnosis.

7.5 Deployment

The application has been tested in the local development environment. Production deployment requires cloud configuration, production servers, database configuration, environment variables, HTTPS, and appropriate file storage.

8. CONCLUSION AND FUTURE WORK

8.1 Conclusion

Milestone 4 focused on integrating the previously developed machine learning components into a complete full-stack healthcare application.

The React frontend and Flask backend were connected through REST APIs, MongoDB was integrated for persistent storage, authentication was implemented, and major application modules were developed.

The resulting MedAssistAI platform combines:

- AI consultation
- Symptom-based disease prediction
- Medical report analysis
- Medical report management
- Image analysis
- Health recommendations
- Appointment management
- Hospital services

The project has therefore progressed from individual ML components toward an integrated AI-assisted healthcare platform.

8.2 Future Work

The next stage of development will focus on:

- Production deployment
- Cloud MongoDB configuration
- Improving model accuracy
- Handling dataset imbalance
- Improved AI response validation
- Cloud file storage
- Enhanced hospital services
- Improved security
- Automated testing
- Performance optimization